

Trabalho Prático – Classificação no CIFAR-10 com VGG16 e Transfer Learning

Artur Ferreira de Castro - 202335029

Caio Machado Bertelli - 22465563B

Livia Ribeiro Pessamilio - 202165088C

Vinicius Macuco Leite Carvalhaes - 202365213AC

19/01/2026

Universidade Federal de Juiz de Fora (UFJF)

Departamento de Ciência da Computação

Disciplina: Visão Computacional (2025/03)

Professor: Prof. Luiz Maurílio da Silva Maciel

Repositório do projeto: <https://github.com/ArturUFJF/VC-Trabalho-Pratico.git>

1 Estratégias de implementação

1.1 Implementação da VGG16 (do zero)

Nesta etapa, foi implementada a arquitetura VGG16 por composição de camadas (convolução, pooling e camadas totalmente conectadas), sem utilizar uma implementação pronta da rede completa. Foram avaliadas variações de hiperparâmetros como *learning rate*, número de épocas e *batch size*. Também foram testadas alterações na cabeça do classificador (camadas totalmente conectadas) para reduzir overfitting, incluindo o uso de dropout.

1.2 Transfer learning

No transfer learning, foram utilizados modelos pré-treinados em ImageNet (VGG16, ResNet50 e DenseNet121), substituindo a camada de classificação para o problema do CIFAR-10. Também foram realizados testes variando hiperparâmetros de treinamento e a quantidade de camadas treináveis por congelamento/descongelamento (fine-tuning).

1.3 Monitoramento com W&B

As execuções (*runs*) foram monitoradas no Weights & Biases (W&B), permitindo comparação sistemática e exportação das métricas consolidadas (CSV), incluindo acurácia global e acurácia por classe.

2 Descrição dos experimentos

2.1 Configuração das máquinas

Os experimentos foram executados em máquinas identificadas nos logs do W&B, todas com Windows 11 e GPUs NVIDIA compatíveis com CUDA. As configurações são:

- **Máquina 1:** Windows 11 (Build 26100), CPU 6C/12T, GPU NVIDIA GeForce RTX 3060 (12 GB VRAM), CUDA 13.0, RAM ~16 GB, Python CPython 3.12.10.
- **Máquina 2:** Windows 11 (Build 26200), CPU 4C/8T, GPU NVIDIA GeForce GTX 1650, CUDA 11.0, RAM ~14 GB, Python CPython 3.12.10.
- **Máquina 3:** Windows 11 (Build 26100), CPU 6C/12T, GPU NVIDIA GeForce RTX 4060 (~8 GB VRAM), CUDA 13.0, RAM ~28 GB, Python CPython 3.12.3.

De forma geral, VGG16/VGG16-TL foram testadas nas três máquinas. A ResNet50-TL foi executada principalmente na Máquina 3 (com mixed precision fp16) e a DenseNet121-TL foi executada em duas etapas: sem upscaling na Máquina 1 e, por limitação de hardware, com upscaling em Google Colab (GPU Tesla T4), portanto os tempos de execução da DenseNet121 com e sem upscaling não são diretamente comparáveis.

A Máquina 2 foi usada apenas para execuções preliminares/curtas de sanity check; os resultados finais reportados (Tabela 1) concentram-se nas Máquinas 1 e 3, e no Colab para DenseNet com upscaling.

2.2 Protocolo de validação

Durante os experimentos, foi utilizado `x_test` como conjunto de validação (isto é, as métricas reportadas como `val_accuracy` e `val_loss` referem-se ao desempenho no conjunto `x_test`).

2.3 Critério de seleção do melhor modelo

Para padronizar a comparação no relatório, definiu-se como “melhor modelo” a execução finalizada (`State = finished`) com maior `best_val_accuracy` no export do W&B. Entretanto, ao longo da experimentação, também se observou a `val_loss` e o comportamento dos gráficos (estabilidade, divergências e sinais de overfitting), o que motivou algumas decisões qualitativas descritas nas subseções seguintes.

2.4 Tabela comparativa dos melhores resultados

A Tabela 1 resume, para cada arquitetura, o melhor resultado por `best_val_accuracy` e os hiperparâmetros correspondentes (extraídos do CSV do W&B).

Tabela 1: Resumo dos melhores resultados por arquitetura (critério: `best_val_accuracy`; apenas runs finished).

Arquitetura	Run (W&B)	best val acc	Learning Rate	Batch	Épocas	Unfrozen
VGG16 (do zero)	vgg16-run-2029	0.8532	0.01	128	30	–
VGG16-TL	vgg16-tl-run-2142	0.9439	0.00001	32	50	12
ResNet50-TL	resnet50-tl-run-1109	0.9491	0.0001	64	50	30
DenseNet121-TL	densenet121-tl-run-0147	0.9342	0.00001	32	20	90

2.5 Processo individual de experimentação

2.5.1 DenseNet121 com transfer learning (com e sem upscaling)

A estratégia adotada para DenseNet121 foi uma busca sistemática a partir de uma configuração base (15 épocas, batch 64, learning rate 1×10^{-3} e 0 camadas descongeladas), com o objetivo de equilibrar acurácia final e custo computacional. Nesta arquitetura, o processo foi dividido em duas fases: (i) treinamento no CIFAR-10 no tamanho original 32×32 e (ii) treinamento com upscaling para 224×224 , compatível com a entrada típica da DenseNet121.

Fase 1: CIFAR-10 32×32 (sem upscaling). Primeiro, testou-se conjuntamente o par (learning rate, camadas descongeladas), selecionando combinações com menor `val_loss` e curvas estáveis de loss/acurácia (evitando divergências). Após a definição de um par estável, variou-se o número de épocas segundo $E_i = 5 \times i$ e selecionou-se o ponto de melhor custo-benefício com base no ganho de acurácia de validação $G_n = A_n - A_{n-1}$. Em seguida, variou-se o batch size segundo $B_i = 32 \times 2^{i-1}$, decidindo entre qualidade e eficiência conforme ganho de tempo e perda de acurácia. Nessa fase, o melhor conjunto encontrado (sem upscaling) ficou em torno de learning rate 1×10^{-5} , 90% das camadas descongeladas, 25 épocas e batch 64, com acurácia de validação média ≈ 0.712 .

Fase 2: CIFAR-10 224×224 (com upscaling). Devido a limitações de hardware, o upscaling foi executado em ambiente distinto (Google Colab, GPU Tesla T4). Mantendo-se a configuração de fine-tuning (learning rate 1×10^{-5} e 90% das camadas descongeladas), observou-se um aumento significativo na acurácia de validação (para ≈ 0.934). Aplicando critérios análogos de otimização, definiu-se 20 épocas como melhor custo-benefício e batch size 32 como escolha final (diferença de tempo pequena e acurácia praticamente igual em batch 32 vs 64, priorizando estabilidade e compatibilidade de memória).

2.5.2 VGG16 com transfer learning (VGG16-TL)

A metodologia de otimização para a VGG16-TL seguiu uma estratégia de descida de coordenadas, avaliando iterativamente subconjuntos de hiperparâmetros mantendo os demais fixos. As imagens foram redimensionadas (upscaling) para 224×224 para compatibilidade com a entrada do modelo pré-treinado.

Passo 1: learning rate e camadas descongeladas. Foram testadas combinações de taxa de aprendizado e número de camadas descongeladas, observando-se que taxas maiores podem gerar instabilidade numérica (divergência, perda NaN) em contexto de transfer

learning. A melhor combinação nesta etapa foi learning rate 1×10^{-5} com 12 camadas descongeladas.

Passo 2: batch size. Com learning rate e camadas descongeladas fixos, variou-se o batch size e escolheu-se o valor que maximizou a acurácia de validação e reduziu a loss de validação, resultando em batch 32.

Passo 3: número de épocas. Foi realizada uma execução estendida até 50 épocas para analisar tendência e sinais de overfitting. Observou-se que 25 épocas apresentaram um bom compromisso entre desempenho e tempo de treinamento (custo-benefício), mas como critério formal do relatório adota-se o modelo com maior `best_val_accuracy`, que ocorreu na run de 50 épocas (Tabela 1).

2.5.3 ResNet50 com transfer learning (ResNet50-TL)

A ResNet50-TL também foi otimizada por descida de coordenadas, com redimensionamento das imagens para 224×224 e uso de mixed precision fp16 para reduzir tempo de treinamento.

Passo 1: learning rate e camadas descongeladas. Testou-se o impacto da taxa de aprendizado e do percentual de camadas descongeladas; valores maiores podem trazer ganhos rápidos de acurácia, mas podem gerar curvas menos suaves. Além do modelo com maior acurácia nessa etapa, também foi observado um modelo com taxa menor (learning rate 1×10^{-6}) com crescimento mais estável, porém máximo menor.

Passo 2: batch size. Com a configuração escolhida do passo anterior, variou-se batch size, sendo escolhido batch 64 por combinar boa acurácia, baixa loss e menor tempo.

Passo 3: número de épocas. Foram realizadas execuções longas (até 50 épocas) e comparadas curvas e valores de `val_loss`. Embora um modelo com learning rate 1×10^{-6} tenha apresentado curvas mais suaves e menor `val_loss` por volta de 25 épocas, o relatório reporta na Tabela 1 a run com maior `best_val_accuracy` (50 épocas, learning rate 1×10^{-4} , 30 camadas descongeladas, batch 64).

2.6 Análise por classe

A análise por classe foi realizada a partir das métricas `class_acc/*` exportadas do W&B. As discussões devem destacar classes com maior e menor acurácia e relacionar com padrões de confusão observados nas matrizes.

3 Matrizes de confusão

As matrizes de confusão abaixo foram exportadas do painel do W&B e são utilizadas como ferramenta qualitativa para identificar padrões de confusão entre classes no CIFAR-10. Como múltiplas execuções foram realizadas por arquitetura, as matrizes não são necessariamente do mesmo run que atingiu a maior `best_val_accuracy`; a seleção do “melhor modelo” é reportada separadamente com base no CSV do W&B.

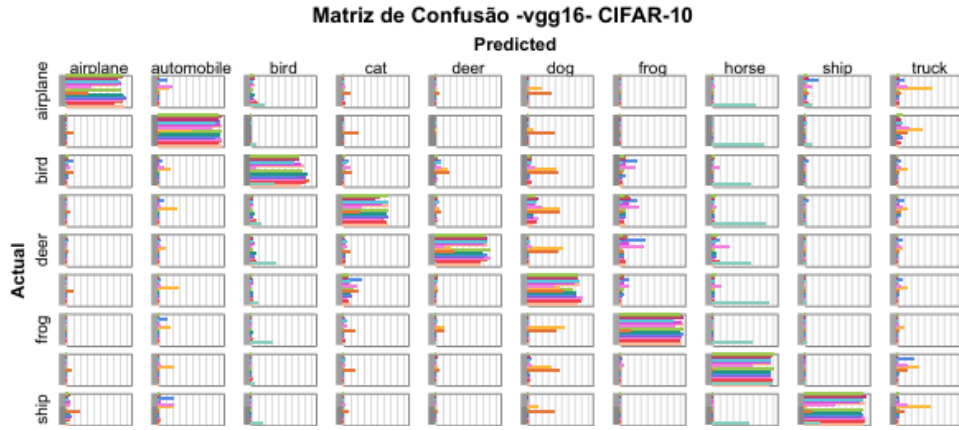


Figura 1: Matriz de confusão (W&B) da VGG16 implementada no CIFAR-10 (validação em `x_test`); usada para análise qualitativa de confusões entre classes.

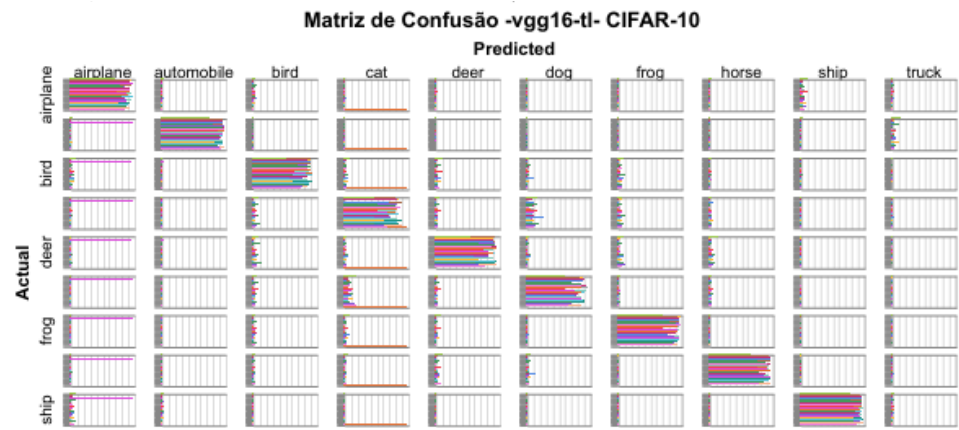


Figura 2: Matriz de confusão (W&B) da VGG16 com transfer learning no CIFAR-10 (validação em `x_test`); usada para análise qualitativa de confusões entre classes.

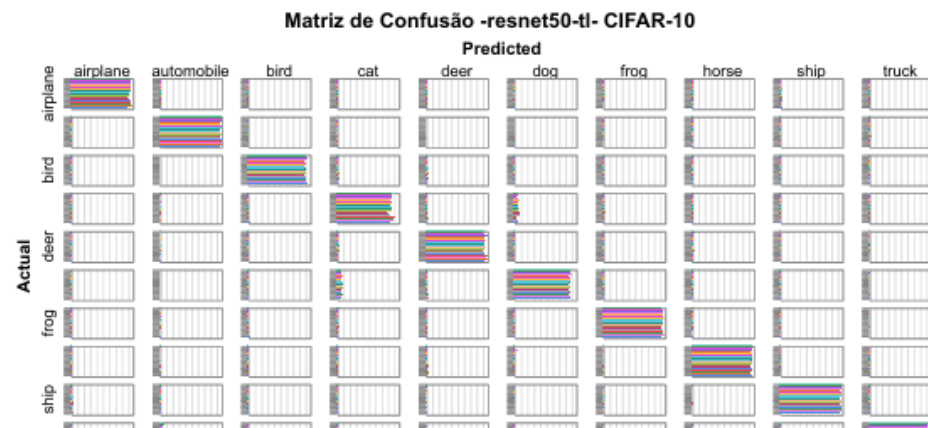


Figura 3: Matriz de confusão (W&B) da ResNet50 com transfer learning no CIFAR-10 (validação em `x_test`); usada para análise qualitativa de confusões entre classes.

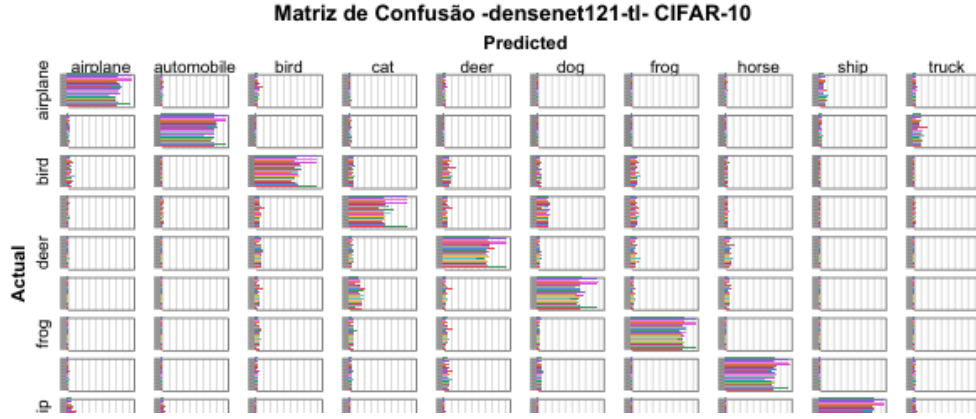


Figura 4: Matriz de confusão (W&B) da DenseNet121 com transfer learning no CIFAR-10 (validação em `x_test`); usada para análise qualitativa de confusões entre classes.

4 Discussão

De forma geral, os modelos com transfer learning obtiveram desempenho superior ao treinamento do zero. Além disso, observou-se que (i) o upscaling para 224×224 impactou fortemente a DenseNet121-TL, (ii) a escolha de camadas descongeladas altera o trade-off entre adaptação ao CIFAR-10 e risco de overfitting, e (iii) o número de épocas deve ser escolhido com base tanto em acurácia quanto em estabilidade de loss e custo computacional.

5 Conclusão

Este trabalho implementou a VGG16 do zero e aplicou transfer learning com VGG16, ResNet50 e DenseNet121 para classificação no CIFAR-10. Os resultados mostraram que modelos com transfer learning obtiveram desempenho superior ao treinamento do zero, e que a escolha de hiperparâmetros (learning rate, batch size, épocas e quantidade de camadas treináveis) impacta diretamente a acurácia e a capacidade de generalização.

Como dificuldades, destacam-se o ajuste de hiperparâmetros, a estabilidade do treinamento em algumas configurações e o tempo de execução dos experimentos. Como trabalhos futuros, recomenda-se testar *data augmentation* mais forte, regularização adicional e estratégias de fine-tuning por blocos para reduzir confusões entre classes visualmente semelhantes.

Referências

- [1] Biewald, L. *Experiment Tracking with Weights and Biases*. 2020. Software disponível em: <https://www.wandb.com/>. Acesso em: 19/01/2026.
- [2] Krizhevsky, A. *Learning Multiple Layers of Features from Tiny Images*. 2009.
- [3] Deng, J. et al. *ImageNet: A Large-Scale Hierarchical Image Database*. CVPR, 2009.